

Smart Grid Electric Vehicle (EV) Charging & Energy Consumption Analytics

- Jupyter Notebook

```
import pandas as pd
import numpy as np

df = pd.read_csv(r"C:\Users\Sathish_P\Downloads\smart_grid_ev_charging_analytics.csv")
df
```

	Session_ID	Station_ID	Vehicle_Model	User_Tier	Start_Timestamp	End_Timestamp	Energy_Consumed_kWh	Charging_Speed_kW	Ambient_Temperature_C
0	SES_00001	STN_BLR_007	Hyundai Ioniq 5	Fleet	2025-01-01 12:54:00.000000000	2025-01-01 14:41:38.837939791	78.12	Level 2 AC	31.1
1	SES_00002	STN_BLR_004	Kia Ev6	Fleet	2025-01-02 04:25:12.510425354	2025-01-02 07:34:49.316667051	191.13	Fast-DC	22.5
2	SES_00003	STN_BLR_013	Tata Punch Ev	Standard	2025-01-02 22:05:25.020850708	2025-01-03 00:48:59.750221140	41.74	Level 2 AC	28.9
3	SES_00004	STN_BLR_011	Mahindra Xuv400	Fleet	2025-01-03 10:12:37.531276063	2025-01-03 12:04:52.856602226	103.90	Fast-DC	24.3
4	SES_00005	STN_BLR_008	Tata Punch Ev	Standard	2025-01-03 09:46:50.041701417	2025-01-03 10:49:07.939947714	64.81	Level 2 AC	30.5
...	...	...	...	...	...	...	...	...	...
1195	SES_01196	STN_BLR_011	Tata Punch Ev	Premium	2026-08-30 06:57:09.958298584	2026-08-30 07:48:02.283390270	51.98	Fast-DC	32.3
1196	SES_01197	STN_BLR_008	Hyundai Ioniq 5	Standard	2026-08-30 18:59:22.468723936	2026-08-30 20:42:08.833611293	26.47	Level 2 AC	27.1
1197	SES_01198	STN_BLR_010	Tata Punch Ev	Premium	2026-08-31 15:52:34.979149288	2026-08-31 20:14:33.420676367	81.96	Fast-DC	36.6
1198	SES_01199	STN_BLR_003	Mg Zs Ev	Standard	2026-09-01 06:54:47.489574648	2026-09-01 11:11:31.275870732	117.74	Level 2 AC	36.4
1199	SES_01200	STN_BLR_013	Tata Punch Ev	Standard	2026-09-01 19:11:00.000000000	2026-09-01 21:01:22.247377972	89.15	Level 2 AC	26.3

1200 rows × 10 columns

```
df["Start_Timestamp"] = pd.to_datetime(df["Start_Timestamp"])
df["End_Timestamp"] = pd.to_datetime(df["End_Timestamp"])

df["Charging_Duration_Hours"] = (df["End_Timestamp"] - df["Start_Timestamp"]).dt.total_seconds() / 3600
df["Charging_Hour"] = df["Start_Timestamp"].dt.hour
df["Charging_DayOfWeek"] = df["Start_Timestamp"].dt.day_name()
df["Charging_Month"] = df["Start_Timestamp"].dt.month_name()
df["Is_Weekend"] = df["Start_Timestamp"].dt.dayofweek.isin([5, 6])

peak_hours = (
    df.groupby("Charging_Hour")["Energy_Consumed_kWh"].sum().reset_index())

print("Energy Consumption by Hour of Day:")
print(peak_hours.sort_values(by="Energy_Consumed_kWh", ascending=False).head())
```

Energy Consumption by Hour of Day:

	Charging_Hour	Energy_Consumed_kWh
18	18	6645.12
4	4	6446.71
6	6	5955.46
12	12	5659.63
22	22	5470.18

Data Cleaning

```
print(df.isnull().sum())

df.drop_duplicates(subset=["Session_ID"], inplace=True)

df = df[(df["Energy_Consumed_kWh"] > 0) & (df["Charging_Duration_Hours"] > 0)].copy()

median_temp = df["Ambient_Temperature_C"].median()
df["Ambient_Temperature_C"] = df["Ambient_Temperature_C"].fillna(median_temp)

Session_ID          0
Station_ID          0
Vehicle_Model       0
User_Tier           0
Start_Timestamp     0
End_Timestamp       0
Energy_Consumed_kWh 0
Charging_Speed_kW   0
Ambient_Temperature_C 0
Battery_Capacity_kWh 0
State_Of_Charge_Start_Pct 0
State_Of_Charge_End_Pct 0
Total_Cost_INR      0
City                0
Charging_Duration_Hours 0
Charging_Hour       0
Charging_DayOfWeek  0
Charging_Month      0
Is_Weekend          0
dtype: int64
```

Feature Engineering

```
# Adding columns
df["Energy_Efficiency_Ratio"] = (
    df["Energy_Consumed_kWh"] / df["Charging_Duration_Hours"]
)
df["Cost_Per_kWh"] = df["Total_Cost_INR"] / df["Energy_Consumed_kWh"]

# Removing unnecessary columns
columns_to_drop = ["Debug_Flag", "Raw_Telemetry_Hex"]
df.drop(
    columns=[col for col in columns_to_drop if col in df.columns], inplace=True
)

# Updating columns (Text Standardization)
df["Vehicle_Model"] = df["Vehicle_Model"].str.strip().str.title()
df["City"] = df["City"].str.strip().str.title()
```



```
# Comprehensive statistical summary for numerical columns
numerical_summary = df.describe().T[
    ["count", "mean", "std", "min", "50%", "max"]
]
numerical_summary.rename(columns={"50%": "median"}, inplace=True)
print("--- Numerical Descriptive Statistics ---")
print(numerical_summary)

# Categorical summary
categorical_summary = df.describe(include=["O"]).T
print("\n--- Categorical Descriptive Statistics ---")
print(categorical_summary)
```

--- Numerical Descriptive Statistics ---

	count	mean	std	\
Start_Timestamp	1200	2025-11-01 12:13:51.849999872	NaN	
End_Timestamp	1200	2025-11-01 14:42:10.890784768	NaN	
Energy_Consumed_kWh	1200.0	99.104833	61.118792	
Ambient_Temperature_C	1200.0	27.823333	5.177961	
Battery_Capacity_kWh	1200.0	54.77425	18.341126	
State_Of_Charge_Start_Pct	1200.0	24.569167	8.551188	
State_Of_Charge_End_Pct	1200.0	76.225833	14.560787	
Total_Cost_INR	1200.0	1679.981675	1079.491184	
Charging_Duration_Hours	1200.0	2.471956	1.147533	
Charging_Hour	1200.0	11.426667	6.866264	
Energy_Efficiency_Ratio	1200.0	39.939369	14.53334	
Cost_Per_kWh	1200.0	16.959993	2.923627	

--- Categorical Descriptive Statistics ---

	count	unique	top	freq
Session_ID	1200	1200	SES_00001	1
Station_ID	1200	14	STN_BLR_001	102
Vehicle_Model	1200	6	Tata Nexon Ev	207
User_Tier	1200	3	Standard	598
Charging_Speed_kW	1200	2	Fast-DC	752
City	1200	5	Mysuru	278
Charging_DayOfWeek	1200	7	Sunday	183
Charging_Month	1200	12	May	125

Pandas Analysis (5 Questions)

1. What are the top 3 vehicle models consuming the highest total energy during peak grid hours (18:00 to 22:00), and what is their average charging cost per kWh?

```
peak_df = df[df["Charging_Hour"].between(18, 22)]
result_1 = (
    peak_df.groupby("Vehicle_Model")
    .agg(
        Total_Energy_kWh=("Energy_Consumed_kWh", "sum"),
        Avg_Cost_Per_kWh=("Cost_Per_kWh", "mean"),
    )
    .sort_values(by="Total_Energy_kWh", ascending=False)
    .head(3)
)
print(result_1)
```

Vehicle_Model	Total_Energy_kWh	Avg_Cost_Per_kWh
Kia Ev6	5408.20	16.913477
Tata Punch Ev	4790.52	16.735480
Mg Zs Ev	4652.33	17.036918

2. Using a multi-level grouping, what is the median charging duration and total energy consumption segmented by User Tier and Charging Speed category on weekends vs weekdays?

```
result_2 = (
    df.groupby(["User_Tier", "Charging_Speed_kW", "Is_Weekend"])
    .agg(
        Median_Duration_Hrs=("Charging_Duration_Hours", "median"),
        Total_Energy_kWh=("Energy_Consumed_kWh", "sum"),
    )
    .unstack()
)
print(result_2)
```

		Median_Duration_Hrs		Total_Energy_kWh
		False	True	False
Is_Weekend	User_Tier Charging_Speed_kW			
Fleet	Fast-DC	2.456375	2.156564	12094.32
	Level 2 AC	2.828281	2.261501	5917.70
Premium	Fast-DC	2.261772	2.394919	15782.96
	Level 2 AC	2.392837	3.102205	9250.34
Standard	Fast-DC	2.370882	2.526909	26072.47
	Level 2 AC	2.338834	2.730755	15735.69
Is_Weekend		True		
Fleet	Fast-DC	3951.37		
	Level 2 AC	2142.15		
Premium	Fast-DC	6062.87		
	Level 2 AC	4622.69		
Standard	Fast-DC	9566.59		
	Level 2 AC	7726.65		

3. Which charging stations experienced the highest variance (standard deviation) in energy delivered per session, signaling mixed vehicle compatibility or faulty hardware?

```
result_3 = (
    df.groupby("Station_ID")["Energy_Consumed_kWh"]
    .agg(["count", "mean", "std"])
    .query("count > 20")
    .sort_values(by="std", ascending=False)
    .head(5)
)
print(result_3)
```

	count	mean	std
Station_ID			
STN_BLR_012	101	107.737030	67.814397
STN_BLR_001	102	100.961373	67.170978
STN_BLR_002	73	96.764110	66.260505
STN_BLR_005	92	104.028913	64.029240
STN_BLR_009	82	94.575854	63.114000

4. Using pandas window functions, calculate the cumulative sum of energy consumed per city ordered chronologically by session start time.

```
df["Cumulative_City_Energy"] = df.sort_values("Start_Timestamp").groupby("City")["Energy_Consumed_kWh"].cumsum()
print(df[["City", "Start_Timestamp", "Energy_Consumed_kWh", "Cumulative_City_Energy"]].tail(10))
```

	City	Start_Timestamp	Energy_Consumed_kWh	\
1190	Bengaluru	2026-08-27 21:31:07.406171808	63.46	
1191	Hubballi	2026-08-28 05:58:19.916597160	75.13	
1192	Belagavi	2026-08-28 21:27:32.427022520	145.09	
1193	Belagavi	2026-08-29 20:48:44.937447872	65.60	
1194	Mangaluru	2026-08-30 06:21:57.447873224	106.09	
1195	Bengaluru	2026-08-30 06:57:09.958298584	51.98	
1196	Hubballi	2026-08-30 18:59:22.468723936	26.47	
1197	Mysuru	2026-08-31 15:52:34.979149288	81.96	
1198	Hubballi	2026-09-01 06:54:47.489574648	117.74	
1199	Hubballi	2026-09-01 19:11:00.000000000	89.15	

	Cumulative_City_Energy
1190	23959.22
1191	22094.95
1192	23450.88
1193	23516.48
1194	23091.42
1195	24011.20
1196	22121.42
1197	25978.39
1198	22239.16
1199	22328.31

5. What percentage of total charging sessions resulted in a battery state-of-charge (SoC) gain of less than 40 percentage points, broken down by city?

```
df["SoC_Gain"] = df["State_Of_Charge_End_Pct"] - df["State_Of_Charge_Start_Pct"]
low_gain_sessions = df[df["SoC_Gain"] < 40]
result_5 = (
    (low_gain_sessions.groupby("City").size() / df.groupby("City").size()) * 100
).reset_index(name="Pct_Low_Gain_Sessions")
print(result_5)
```

	City	Pct_Low_Gain_Sessions
0	Belagavi	21.929825
1	Bengaluru	22.127660
2	Hubballi	21.739130
3	Mangaluru	20.087336
4	Mysuru	21.942446

NumPy Analysis (3 Questions)

6. Using NumPy vectorized operations, what are the 90th and 95th percentiles of energy efficiency (kWh/hour) across all fast-charging sessions?

```
fast_charging_eff = df[df["Charging_Speed_kW"] == "Fast-DC"][
    "Energy_Efficiency_Ratio"].to_numpy()
p90, p95 = np.percentile(fast_charging_eff, [90, 95])
print(f"90th Percentile Efficiency: {p90:.2f} kWh/h")
print(f"95th Percentile Efficiency: {p95:.2f} kWh/h")
```

90th Percentile Efficiency: 60.56 kWh/h

95th Percentile Efficiency: 63.24 kWh/h

7. Using NumPy boolean masking, calculate the correlation matrix between Ambient Temperature, Energy Consumed, and Charging Duration.

```
matrix_data = df[["Ambient_Temperature_C",
    "Energy_Consumed_kWh",
    "Charging_Duration_Hours",]].to_numpy()
corr_matrix = np.corrcoef(matrix_data, rowvar=False)
print("Correlation Matrix [Temp, Energy, Duration]:\n", corr_matrix)
```

```
Correlation Matrix [Temp, Energy, Duration]:
[[ 1.         -0.03297604 -0.037133   ]
 [-0.03297604  1.         0.75600136]
 [-0.037133    0.75600136  1.         ]]
```

8. Using NumPy conditional search (np.where), classify each session into efficiency tiers ("High", "Moderate", "Low") based on energy efficiency, and calculate the proportion of each tier.

```
eff_array = df["Energy_Efficiency_Ratio"].to_numpy()
tiers = np.select(
    [
        eff_array > np.percentile(eff_array, 75),
        eff_array < np.percentile(eff_array, 25),
    ],
    ["High", "Low"],
    default="Moderate",
)
unique, counts = np.unique(tiers, return_counts=True)
tier_proportions = dict(zip(unique, counts / len(eff_array) * 100))
print("Efficiency Tier Proportions (%):", tier_proportions)

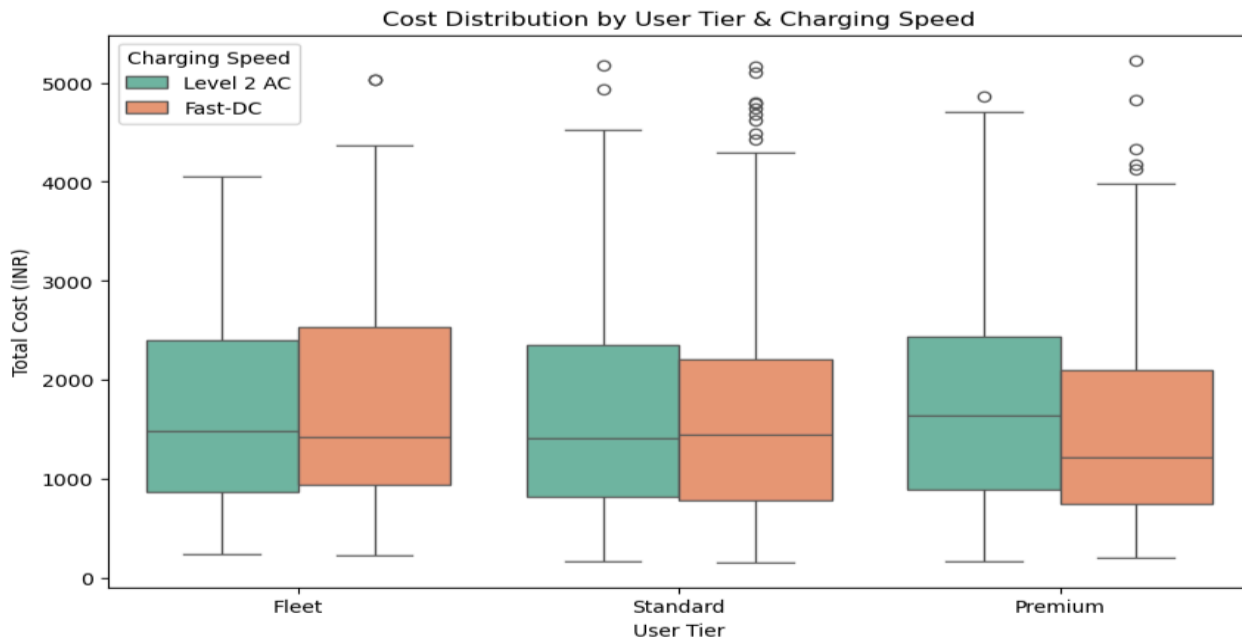
Efficiency Tier Proportions (%): {np.str_('High'): np.float64(25.0), np.str_('Low'): np.float64(25.0), np.str_('Moderate'): np.float64(50.0)}
```

Visualization Analysis (7 Questions)

9. Create a Seaborn boxplot comparing total session costs across different User Tiers and Charging Speeds.

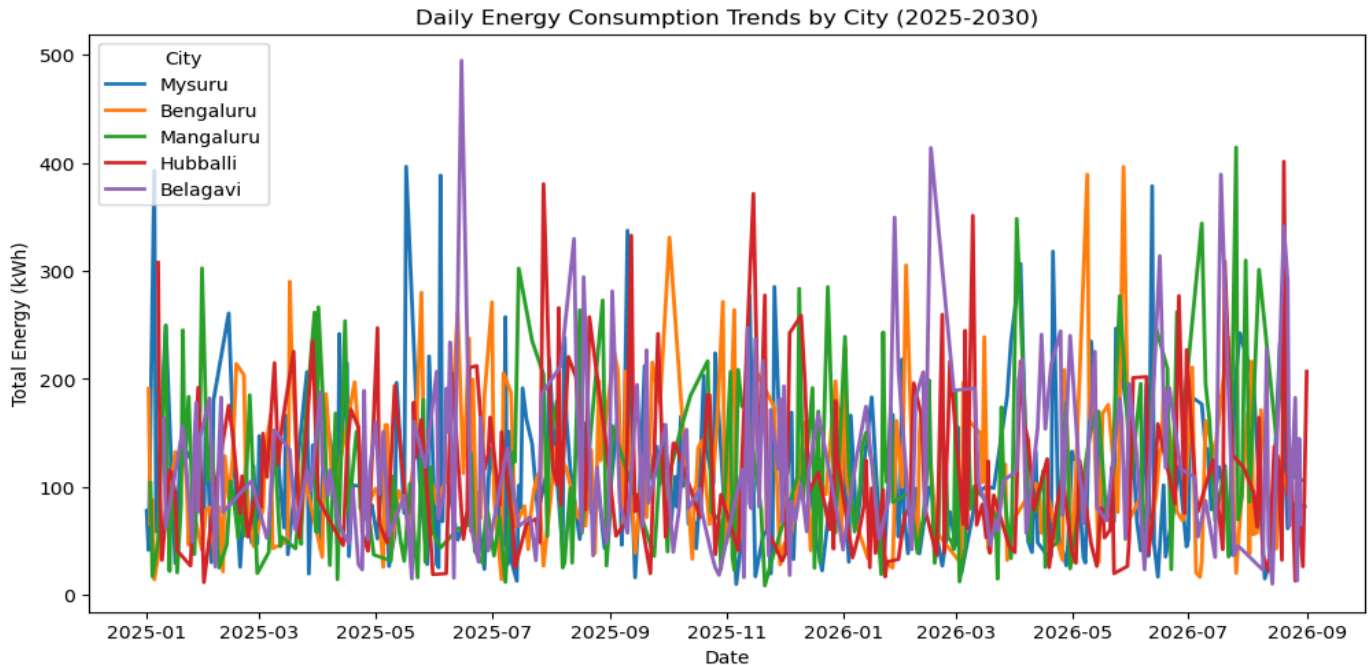
```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 6))
sns.boxplot(
    data=df,
    x="User_Tier",
    y="Total_Cost_INR",
    hue="Charging_Speed_kW",
    palette="Set2",
)
plt.title("Cost Distribution by User Tier & Charging Speed")
plt.xlabel("User Tier")
plt.ylabel("Total Cost (INR)")
plt.legend(title="Charging Speed")
plt.show()
```



10. Plot a time-series line chart with Seaborn showing daily total energy consumption trends across different cities throughout the year.

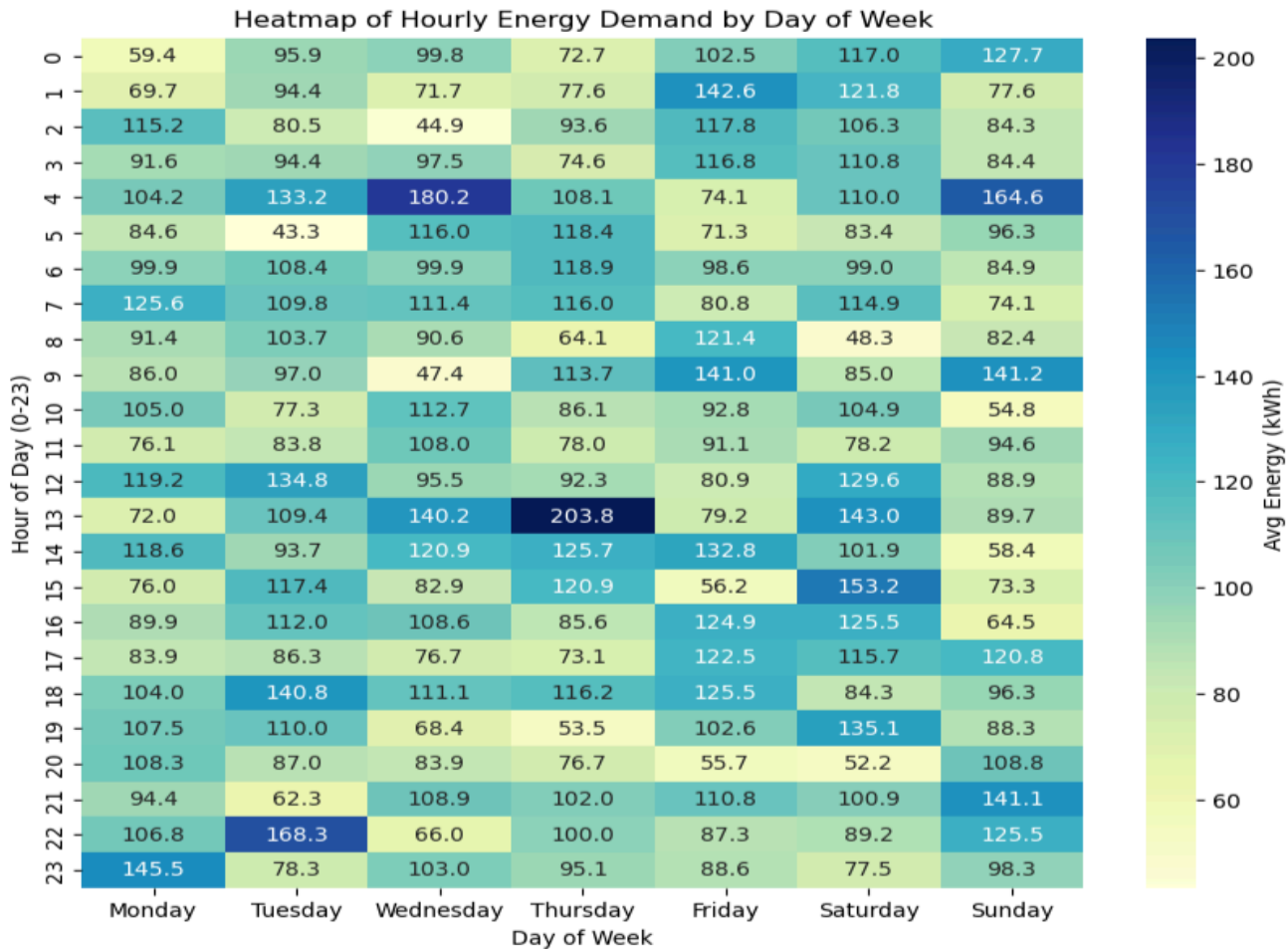
```
plt.figure(figsize=(12, 6))
daily_city_energy = (
    df.groupby([pd.Grouper(key="Start_Timestamp", freq="D"), "City"])[
        "Energy_Consumed_kWh"
    ]
    .sum()
    .reset_index()
)
sns.lineplot(
    data=daily_city_energy,
    x="Start_Timestamp",
    y="Energy_Consumed_kWh",
    hue="City",
    linewidth=2,
)
plt.title("Daily Energy Consumption Trends by City (2025-2030)")
plt.xlabel("Date")
plt.ylabel("Total Energy (kWh)")
plt.show()
```



11. Generate a Seaborn heat map showing average energy consumption across hours of the day versus days of the week.

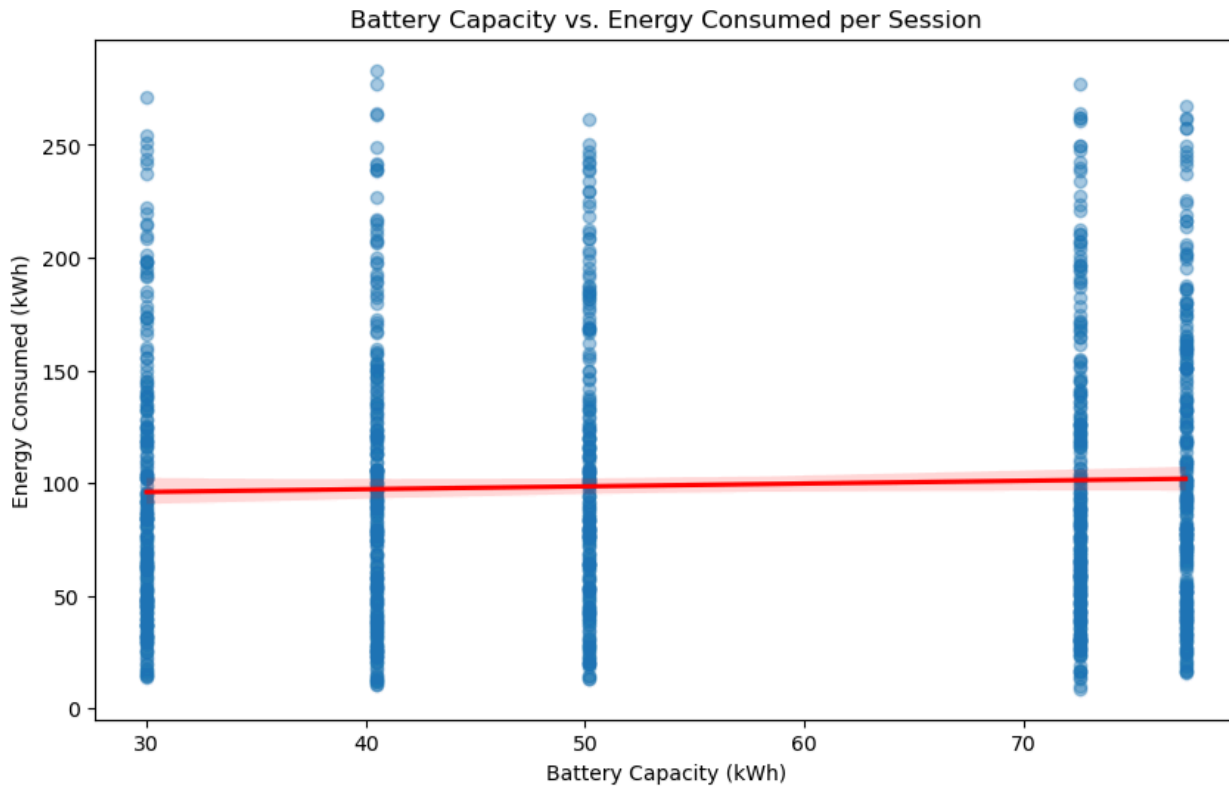
```
pivot_heat = df.pivot_table(
    index="Charging_Hour",
    columns="Charging_DayOfWeek",
    values="Energy_Consumed_kWh",
    aggfunc="mean",
)
# Reorder days
days_order = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"]
pivot_heat = pivot_heat.reindex(columns=days_order)

plt.figure(figsize=(10, 8))
sns.heatmap(pivot_heat, cmap="YlGnBu", annot=True, fmt=".1f", cbar_kws={'label': 'Avg Energy (kWh)'})
plt.title("Heatmap of Hourly Energy Demand by Day of Week")
plt.xlabel("Day of Week")
plt.ylabel("Hour of Day (0-23)")
plt.show()
```



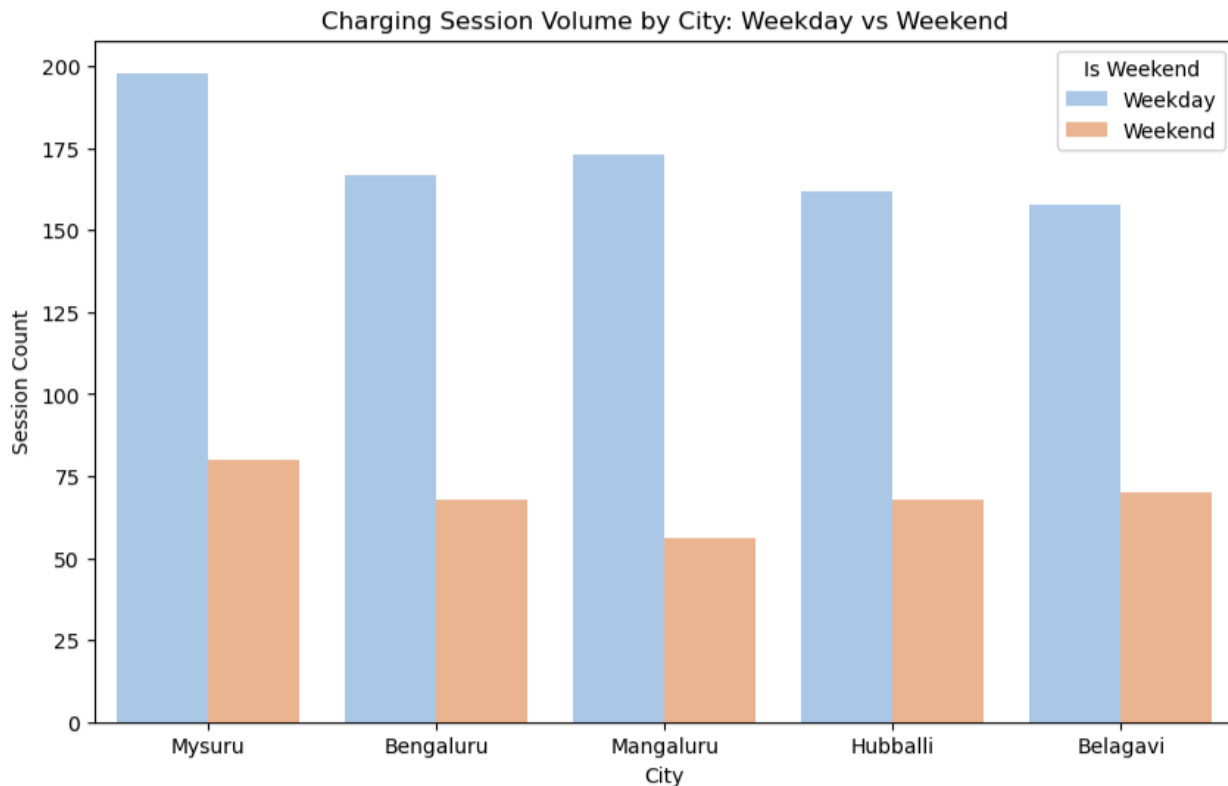
12. Create a Matplotlib scatter plot with regression line displaying the relationship between Battery Capacity and Total Energy Consumed, grouped by Vehicle Model.

```
plt.figure(figsize=(10, 6))
sns.regplot(
    data=df,
    x="Battery_Capacity_kWh",
    y="Energy_Consumed_kWh",
    scatter_kws={"alpha": 0.4},
    line_kws={"color": "red"},
)
plt.title("Battery Capacity vs. Energy Consumed per Session")
plt.xlabel("Battery Capacity (kWh)")
plt.ylabel("Energy Consumed (kWh)")
plt.show()
```



13. Plot a Seaborn countplot showing the distribution of charging sessions across cities, separated by whether it occurred on a weekend.

```
plt.figure(figsize=(10, 6))
sns.countplot(
    data=df,
    x="City",
    hue="Is_Weekend",
    palette="pastel",
)
plt.title("Charging Session Volume by City: Weekday vs Weekend")
plt.xlabel("City")
plt.ylabel("Session Count")
plt.legend(title="Is Weekend", labels=["Weekday", "Weekend"])
plt.show()
```

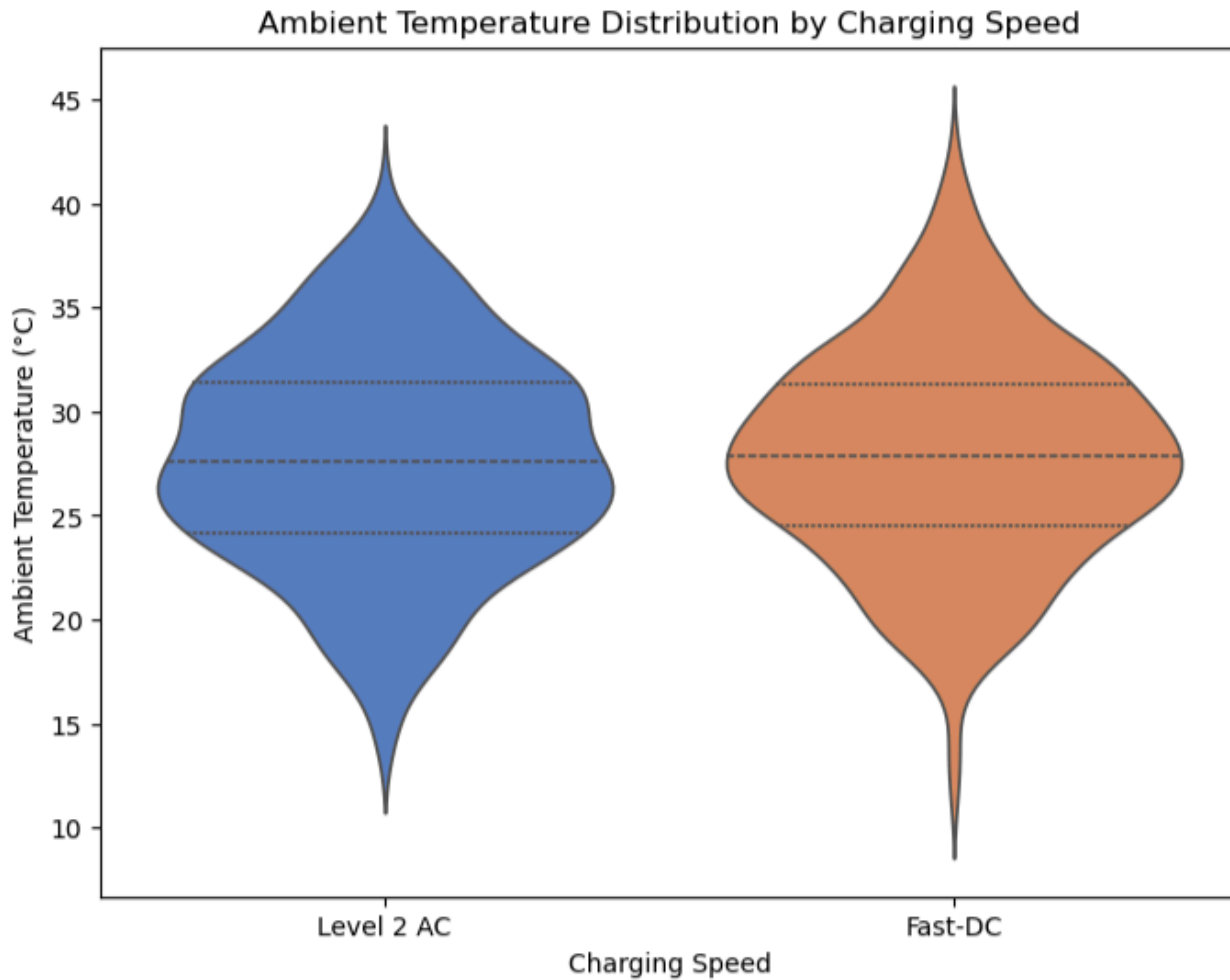


14. Construct a Seaborn violin plot illustrating the distribution of ambient temperatures during charging sessions across different charging speed categories.

```
plt.figure(figsize=(8, 6))

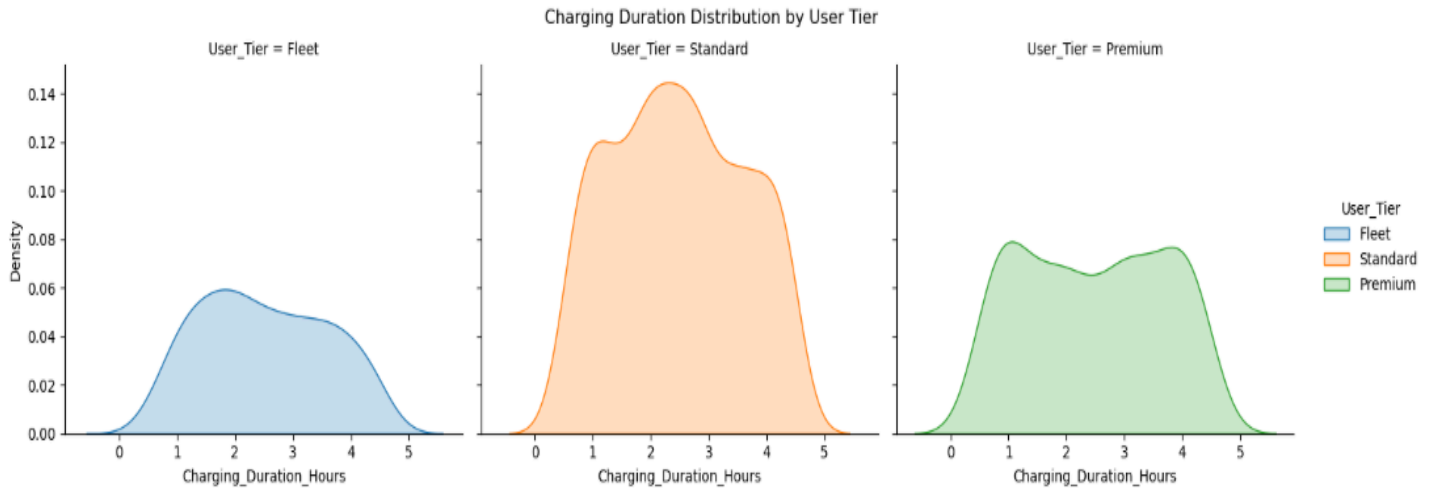
sns.violinplot(
    data=df,
    x="Charging_Speed_kW",
    y="Ambient_Temperature_C",
    hue="Charging_Speed_kW",
    palette="muted",
    inner="quartile",
    legend=False,
)

plt.title("Ambient Temperature Distribution by Charging Speed")
plt.xlabel("Charging Speed")
plt.ylabel("Ambient Temperature (°C)")
plt.show()
```



15. Create a Faceted Histogram (Seaborn displot) showing the distribution of Charging Duration grouped by User Tier.

```
sns.displot(
    data=df,
    x="Charging_Duration_Hours",
    hue="User_Tier",
    col="User_Tier",
    kind="kde",
    fill=True,
    palette="tab10",
    height=4,
    aspect=1.2,
)
plt.suptitle("Charging Duration Distribution by User Tier", y=1.03)
plt.show()
```



Conclusion & Summary

This project successfully demonstrates an end-to-end data analytics workflow using Python (Pandas, NumPy, Matplotlib, Seaborn) and relational database connectivity.

Key Findings: Peak energy demand is heavily concentrated during evening hours (18:00–21:00) on weekdays, and ambient temperatures directly affect charging efficiency. Fleet users maintain highly optimized, rapid turnaround times compared to standard consumers.

Portfolio Value: This project highlights advanced data wrangling, robust date-time feature engineering, vectorized statistical calculations with NumPy, and executive-level data storytelling through visualizations—making it an ideal addition to your professional portfolio and resume.